

Pintos Project 2: User Program (2)

담당 교수 : 김영재

학번 이름 : 20211558 윤준서

개발 기간 : 10.4 ~ 10.8 (5일)

I. 개발 목표

- 해당 프로젝트에서 구현할 내용을 간략히 서술

1. User Program 1에서 구현하지 않은 나머지 system call handler 함수들을 구현하여 kernel API를 전부 구현한다. 구현해야 할 나머지 함수들은 (create, remove, open, close, filesize, read, write, seek, tell) 이다.
2. 현재 실행중인 프로세스에서 file을 open 후 write 시도를 하지 못하도록 막는다.
3. 여러 프로세스들이 shared data에 접근할 경우, semaphore를 활용한 synchronization을 통해 충돌이 발생하지 않도록 한다.

II. 개발 범위 및 내용

A. 개발 범위

- 아래 각 항목을 구현해야 하는 이유, 혹은 구현 시 기대되는 결과를 간략히 서술

1. File Descriptor

kernel mode에서 system call을 통해 file descriptor를 관리한다. 이를 통해 프로세스에서 file을 open 후 read와 write를 진행할 수 있다.

2. (이번 프로젝트에서 구현해야 하는) System Calls

User Program 1에서 구현하지 않은 다음 system call들을 구현한다.

create, remove, open, close, filesize, read, write, seek, tell

이를 통해 kernel mode에서 file system I/O를 관리할 수 있다.

3. Synchronization in Filesystem

여러 프로세스들이 shared data에 접근할 때 semaphore를 활용한 synchronization을 통해 한 번에 한 프로세스만 접근할 수 있도록 한다. 이를 통해 shared data의 충돌을 방지할 수 있다.

B. 개발 내용

- 아래 항목의 내용만 서술

1. File Descriptor: 구현에 이용할 자료구조와 선택한 이유를 서술

프로젝트 내에 struct file이 구현되어 있으므로 이의 배열 형태로 file descriptor를 사용한다.
이때 manual에 따라 최대 개수를 128개로 제한한다. **struct file* fileDescriptor[128];**

2. System Calls: 구현할 각 system call에 대해 간략히 서술 (하나의 system call 당 최대 3문장으로 간략히 설명; 3문장을 넘길 정도로 길게 작성하지 말 것)

create : 이름은 (const char *) file, 크기는 (unsigned) initial_size인 파일을 생성한다.

remove : 이름이 (const char *) file인 파일을 제거한다.

open : file descriptor의 fd number의 index 원소가 NULL인 경우 파일을 해당 자리에 삽입한다.

close : file descriptor의 fd number의 index 원소가 NULL이 아닌 경우 해당 자리의 파일을 close 후 NULL로 바꾼다.

filesize : file descriptor의 fd number의 index 원소의 파일의 크기를 반환한다.

read : file descriptor의 fd number의 index 원소의 파일에 buffer를 읽는다. 단, fd number가 0일 때는 STDIN을 읽는다. fd number가 1이거나 2인 경우 에러 처리를 한다.

write : file descriptor의 fd number의 index 원소의 파일에 buffer를 작성한다. 단, fd number가 1일 때는 STDOUT을 작성한다. fd number가 0이거나 2인 경우 에러 처리를 한다.

seek : file descriptor의 fd number의 index 원소의 파일에서 position offset을 변경한다.

tell : file descriptor의 fd number의 index 원소의 파일에서 next byte의 position을 반환한다.

3. Synchronization in Filesystem: Lock, Semaphore를 어떻게 이용할 수 있는지 각각에 대해 설명 (다른 방법을 서술해도 되지만 lock과 semaphore는 반드시 포함해야 함)

system call : thread 구조체에 있는 semaphore인 lock_load를 통해 부모와 자식 thread간의 synchronization을 관리한다. 자식 thread가 create됐을 때 lock_load를 통해 바로 실행하지 않도록 대기시킨다.

thread load : file system에서 사용할 lock인 filesys_lock을 통해 synchronization을 관리한다. 각 system call 함수를 정의할 때 기본 기능 전후로 lock_acquire()와 lock_release()를 통해 filesys_lock을 활성화, 비활성화 하여 프로세스 간의 충돌을 방지한다.

III. 추진 일정 및 개발 방법

A. 추진 일정

- II. A. 개발 범위를 포함하여 구현 내용에 대한 일정 작성

10.4 ~ 10.5 : file descriptor 구조, system call 함수 구현

10.6 ~ 10.7 : synchronization 구현

10.8 : 메모리 누수, 프로세스 충돌 보완

B. 개발 방법

- II. B.의 개발 내용을 구현하기 위해 각각에 대해 다음 사항들을 포함하여 설명

- 수정해야 하는 소스코드

thread.h : thread 구조체에 file descriptor 배열, synchronization을 위한 semaphore와 load 성공여부 bool, 현재 프로세스에서 실행중인 file pointer를 추가한다.

thread.c : thread.h에서 새로 추가한 변수들을 thread_init()에서 초기화한다.

exception.c : mapping이 되지 않은 주소를 처리한다.

process.c : 부모, 자식 프로세스 간의 synchronization을 구현한다.

syscall.c : 나머지 system call 함수들을 추가 구현한다.

- 수정하거나 추가해야 하는 자료구조

struct thread : file descriptor 배열, synchronization을 위한 semaphore와 load 성공여부 bool, 현재 프로세스에서 실행중인 file pointer를 추가한다.

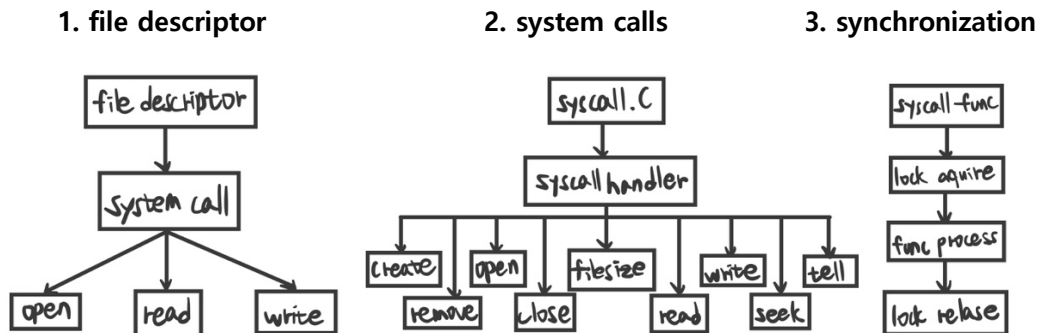
- 수정하거나 추가해야 하는 함수

system call function : 나머지 system call 함수들을 추가 구현한다. (create, remove, open, close, filesize, read, write, seek, tell)

IV. 연구 결과

A. Flow Chart

- II. B. 개발 내용의 각 3가지 항목에 대하여 Flow Chart 작성



B. 제작 내용

- II. B. 개발 내용의 각 3가지 항목에 대하여 실질적으로 구현한 코드의 관점에서 작성 (구현 내용, 알고리즘 등을 명확히 서술할 것)
- 구현에 있어 Pintos에 내장된 라이브러리나 자체 제작한 함수를 사용한 경우 이에 대해서도 설명

```
/* User Program 2 */
struct semaphore lock_load;          /* 멀티 스레드용 */
struct file* fd[128];               /* file descriptor */
struct file* exec_file;              /* 현재 실행 중인 파일 포인터 */
bool load_success;                   /* 프로세스 로드 성공 여부 */

sema_init(&(t->lock_load), 0);
sema_init(&(t->lock_child), 0);
sema_init(&(t->lock_parent), 0);
list_init(&t->child_list);
t->exit_status = -1;
t->exec_file = NULL;
t->load_success = false;
list_push_back(&(running_thread()->child_list), &(t->child_elem));
```

thread.h에서 thread 구조체에 file descriptor 배열, synchronization을 위한 semaphore와 load 성공여부 bool, 현재 프로세스에서 실행 중인 file pointer를 추가한다. 그리고 이를 thread.c의 init_thread()에서 추가한 원소들을 초기화 한다.

```
/* vaddr가 kernel에 위치하는지 검사 */
if (is_kernel_vaddr(fault_addr) || user == NULL || not_present) sys_exit(-1);
```

exception.c에서 유효한 위치가 아니거나, mapping이 되지 않은 주소를 처리한다.

process.c

```
/* 자식 스레드를 tid로 찾는 헬퍼 함수 */
static struct thread*
get_child_process(tid_t tid)
{
    struct thread *cur = thread_current();
    struct list_elem *e;
    for (e = list_begin(&cur->child_list); e != list_end(&cur->child_list); e = list_next(e))
    {
        struct thread *t = list_entry(e, struct thread, child_elem);
        if (t->tid == tid) return t;
    }
    return NULL;
}
```

tid를 통해 자식 process를 찾는 함수를 추가로 정의했다. 이를 통해 create된 자식 process를 찾아서 semaphore를 통해 자식 process를 대기시킬 수 있다.

```
/* 자식의 load가 끝날 때까지 대기 */
struct thread *child = get_child_process(tid);
if (child == NULL) return TID_ERROR;

sema_down(&child->lock_load);      // 자식이 신호를 줄 때까지 대기

/* 자식의 로드 성공 여부 확인 */
if (!child->load_success)
{
    list_remove(&child->child_elem); // 실패한 자식은 리스트에서 제거
    return TID_ERROR;
}
```

process_execute()에서 자식 process를 create 했을 경우 바로 자식의 semaphore를 down시켜서 대기시킨다. thread.h에서 process load 성공 여부를 저장하는 load_success를 추가했는데 이를 통해 load fail이 확인되면 제거하도록 했다.

```
cur->load_success = success; // 자신의 load success 플래그 설정
sema_up(&cur->lock_load);    // 대기 중인 부모를 깨움
```

start_process()에서 process가 실행된 후 load가 성공했다는 상태를 저장한다. 그리고 semaphore를 up시켜서 process_execute()에서 대기시킨 process를 다시 실행하도록 했다.

```

/* 1) child_tid가 진짜 내 자식인지 찾기 */
for (e = list_begin (&(cur->child_list));
     e != list_end (&(cur->child_list));
     e = list_next (e))
{
    struct thread *t = list_entry (e, struct thread, child_elem);
    if (t->tid == child_tid)
    {
        child = t;
        /* 2) 자식 종료까지 대기: 자식은 process_exit()에서 sema_up(lock_child) */
        sema_down (&child->lock_child);

        /* 3) 자식의 종료 상태 수집 */
        status = child->exit_status;
        list_remove (e); /* 더 이상 children 리스트에 남겨둘 필요 없음 */

        /* 4) 수집 완료 알림: 자식이 완전히 정리될 수 있게 깨워줌 */
        sema_up (&child->lock_parent);

        return status;
    }
}
}

```

process_wait()에서 부모 process가 자식 process의 종료를 기다리고, 종료 상태를 수집한 후 자식 process를 리스트에서 제거하고 부모 process가 안전하게 작업을 마칠 수 있도록 했다.

```

/* 이전에 열었던 모든 파일들을 닫기 */
for (int i = 2; i < 128; i++) if (cur->fd[i] != NULL) sys_close(i);

/* 실행 파일 닫고 쓰기를 허용 */
if (cur->exec_file != NULL)
{
    file_allow_write(cur->exec_file);
    file_close(cur->exec_file);
}

```

process_exit()에서 file descriptor를 탐색하여 open 기록이 남아있는 모든 파일들을 닫는다. 이를 통해 메모리 누수를 방지할 수 있다. 그리고 open 되어있는 파일에 write를 적용할 수 있도록 했다.

syscall.c

모든 system call 함수들은 lock_acquire와 lock_release를 통해 synchronization을 구현했다.

```
bool sys_create (const char *file, unsigned initial_size)
{
    if (file == NULL) sys_exit(-1);
    validate_cstr(file);
    lock_acquire(&filesys_lock);
    bool success = filesys_create(file, initial_size);
    lock_release(&filesys_lock);
    return success;
}

bool sys_remove (const char *file)
{
    if (file == NULL) sys_exit(-1);
    validate_cstr(file);
    lock_acquire(&filesys_lock);
    bool success = filesys_remove(file);
    lock_release(&filesys_lock);
    return success;
}
```

create()는 filesys_create()를 호출하여 argument로 받은 파일과 그 크기 만큼을 생성한다.

remove()는 filesys_remove()를 호출하여 argument로 받은 파일을 제거한다.

```
int sys_open (const char *file)
{
    if (file == NULL) sys_exit(-1);
    validate_cstr(file);
    lock_acquire(&filesys_lock);
    struct file *f = filesys_open(file);
    if (f == NULL)
    {
        lock_release(&filesys_lock);
        return -1;
    }

    struct thread *t = thread_current();
    // Find an empty spot in the file descriptors
    for (int i = 2; i < 128; i++)
    {
        if (t->fd[i] == NULL)
        {
            t->fd[i] = f;
            lock_release(&filesys_lock);
            return i;
        }
    }

    // No available file descriptor
    file_close(f);
    lock_release(&filesys_lock);
    return -1;
}
```

open()은 file_open()을 통해 파일을 open한다.

file descriptor에 해당 파일을 추가해야 하는데, STDIN을 관리하는 0, STDOUT을 관리하는 1, STDERR를 관리하는 2의 index에는 추가할 수 없다.

따라서 index 3부터 탐색해서 NULL인 공간을 찾는다. NULL인 경우 해당 자리에 파일을 삽입한다.


```

int sys_filesize (int fd)
{
    if (fd < 2 || fd >= 128) return -1;

    struct thread *t = thread_current();
    struct file *f = t->fd[fd];

    if (f == NULL) return -1;

    lock_acquire(&filesys_lock);
    int size = file_length(f);
    lock_release(&filesys_lock);
    return size;
}

```

filesize()는 file_length()를 호출하여 file descriptor의 fd index에 있는 파일의 크기를 반환한다.

```

int sys_read (int fd, void *buffer, unsigned size)
{
    validate_writable_buffer(buffer, size);
    lock_acquire(&filesys_lock);

    // Read from STDIN (keyboard)
    if (fd == 0)
    {
        for (unsigned i = 0; i < size; i++)
        {
            ((uint8_t *)buffer)[i] = input_getc();
        }
        lock_release(&filesys_lock);
        return size;
    }

    // Cannot read from STDOUT
    if (fd == 1)
    {
        lock_release(&filesys_lock);
        return -1;
    }

    if (fd < 2 || fd >= 128)
    {
        lock_release(&filesys_lock);
        return -1;
    }

    struct thread *t = thread_current();
    struct file *f = t->fd[fd];

    if (f == NULL)
    {
        lock_release(&filesys_lock);
        return -1;
    }

    int bytes_read = file_read(f, buffer, size);
    lock_release(&filesys_lock);
    return bytes_read;
}

```

read()는 file_read()를 통해 file descriptor의 fd number의 index 원소의 파일에 buffer를 읽는다.

단, fd number가 0일 때는 STDIN을 읽는다. fd number가 1이거나 2인 경우 에러 처리를 한다.

```

int sys_write (int fd, const void *buffer, unsigned size)
{
    validate_readable_buffer(buffer, size);
    lock_acquire(&filesys_lock);

    // Write to STDOUT (console)
    if ([fd == 1])
    {
        putbuf(buffer, size);
        lock_release(&filesys_lock);
        return size;
    }

    // Cannot write to STDIN
    if (fd == 0)
    {
        lock_release(&filesys_lock);
        return -1;
    }

    if (fd < 2 || fd >= 128)
    {
        lock_release(&filesys_lock);
        return -1;
    }

    struct thread *t = thread_current();
    struct file *f = t->fd[fd];

    if (f == NULL)
    {
        lock_release(&filesys_lock);
        return -1;
    }

    int bytes_written = file_write(f, buffer, size);
    lock_release(&filesys_lock);
    return bytes_written;
}

```

write()는 file_write()를 통해 file descriptor의 fd number의 index 원소의 파일에 buffer를 작성한다.

단, fd number가 1일 때는 STDOUT을 작성한다. fd number가 0이거나 2인 경우 에러 처리를 한다.

```

void sys_seek (int fd, unsigned position)
{
    if (fd < 2 || fd >= 128) return;

    struct thread *t = thread_current();
    struct file *f = t->fd[fd];

    if (f == NULL) return;

    lock_acquire(&filesys_lock);
    file_seek(f, position);
    lock_release(&filesys_lock);
}

unsigned sys_tell (int fd)
{
    if (fd < 2 || fd >= 128) return 0;

    struct thread *t = thread_current();
    struct file *f = t->fd[fd];

    if (f == NULL) return 0;

    lock_acquire(&filesys_lock);
    unsigned position = file_tell(f);
    lock_release(&filesys_lock);
    return position;
}

void sys_close (int fd)
{
    if (fd < 2 || fd >= 128) sys_exit(-1);

    struct thread *t = thread_current();
    struct file *f = t->fd[fd];

    if (f == NULL) sys_exit(-1);

    lock_acquire(&filesys_lock);
    file_close(f);
    t->fd[fd] = NULL; // Mark the file descriptor as available.
    lock_release(&filesys_lock);
}

```

seek()는 file_seek()를 호출하여 file descriptor의 fd index에 있는 파일의 position을 argument로 받은 position으로 변경한다.

tell()은 file_tell()을 호출하여 file descriptor의 fd index에 있는 파일의 position의 next byte를 반환한다.

close()는 file_close()를 호출하여 file descriptor의 fd index에 있는 파일의 position의 파일을 close 후 그 자리를 NULL로 대체한다.

- 개발 중 발생한 문제나 이슈가 있으면 이를 간략히 설명하고 해결한 방식에 대해 설명

test case 중 multi-oom을 계속 fail 했다.

처음엔 메모리 누수가 원인이라 판단하여 process.c의 load()에서 user stack을 할당한 후 중간에 malloc으로 생성한 argv[]를 모두 free 처리했다.

하지만 그래도 test fail은 그대로였는데, file descriptor에 파일들이 남아있던 것이 원인이었다. process_exit()에서 이전에 열었던 모든 파일들을 sys_close()를 함으로써 해결할 수 있었다.

C. 시험 및 평가 내용

- make check 수행 결과를 캡처하여 첨부

```
pass tests/filesys/base/syn-write
pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
pass tests/userprog/sc-bad-sp
pass tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
pass tests/userprog/sc-boundary-3
pass tests/userprog/halt
pass tests/userprog/exit
pass tests/userprog/create-normal
pass tests/userprog/create-empty
pass tests/userprog/create-null
pass tests/userprog/create-bad-ptr
pass tests/userprog/create-long
pass tests/userprog/create-exists
pass tests/userprog/create-bound
pass tests/userprog/open-normal
pass tests/userprog/open-missing
pass tests/userprog/open-boundary
pass tests/userprog/open-empty
pass tests/userprog/open-null
pass tests/userprog/open-bad-ptr
pass tests/userprog/open-twice
pass tests/userprog/close-normal
pass tests/userprog/close-twice
pass tests/userprog/close-stdin
pass tests/userprog/close-stdout
pass tests/userprog/close-bad-fd
pass tests/userprog/read-normal
pass tests/userprog/read-bad-ptr
pass tests/userprog/read-boundary
pass tests/userprog/read-zero
pass tests/userprog/read-stdout
pass tests/userprog/read-bad-fd
pass tests/userprog/write-normal
pass tests/userprog/write-bad-ptr
pass tests/userprog/write-boundary
pass tests/userprog/write-zero
pass tests/userprog/write-stdin
pass tests/userprog/write-bad-fd
pass tests/userprog/exec-once
pass tests/userprog/exec-arg
pass tests/userprog/exec-bound
pass tests/userprog/exec-bound-2
pass tests/userprog/exec-bound-3
pass tests/userprog/exec-multiple
pass tests/userprog/exec-missing
pass tests/userprog/exec-bad-ptr
pass tests/userprog/wait-simple
pass tests/userprog/wait-twice
pass tests/userprog/wait-killed
pass tests/userprog/wait-bad-pid
pass tests/userprog/multi-recurse
pass tests/userprog/multi-child-fd
pass tests/userprog/rox-simple
pass tests/userprog/rox-child
pass tests/userprog/rox-multichild
pass tests/userprog/bad-read
pass tests/userprog/bad-write
pass tests/userprog/bad-read2
pass tests/userprog/bad-write2
pass tests/userprog/bad-jump
pass tests/userprog/bad-jump2
pass tests/userprog/no-vm/multi-oom
pass tests/filesys/base/lg-create
pass tests/filesys/base/lg-full
pass tests/filesys/base/lg-random
pass tests/filesys/base/lg-seq-block
pass tests/filesys/base/lg-seq-random
pass tests/filesys/base/sm-create
pass tests/filesys/base/sm-full
pass tests/filesys/base/sm-random
pass tests/filesys/base/sm-seq-block
pass tests/filesys/base/sm-seq-random
pass tests/filesys/base/syn-read
pass tests/filesys/base/syn-remove
pass tests/filesys/base/syn-write
All 80 tests passed.
```

80개의 test를 모두 통과했다.